



Venus Internal Security Review

Smart Contract Security

Hub-Funded Spoke Pools Security Review Report

Venus Protocol

About this review

This is an **internal** review produced by the Venus team. It is not a third-party audit and carries no external assurance.

The findings were generated by running the two security skills in [venus-ai-toolkit/skills/security](#) over the spoke pool contracts, and then verifying every result by hand against the source.

Source	What it contributed
<code>solidity-auditor</code> , 12 agents in parallel	The findings and the informational leads
<code>x-ray</code>	Unenforced properties, test gaps, churn analysis
Manual review of the 5 spoke deploy scripts	L-04. Both skills are Solidity-only and do not read <code>.ts</code>

How to read a finding

Every finding carries a **Status**:

- **Fixed**: a change was written, tested and merged. The commit is linked.
- **ACK**: the behaviour is real but accepted. The reason is stated in full.
- **Open**: no decision taken yet.

Each agent result was re-derived from the source before being recorded. Several agent claims were rejected on that check and are not listed; several others survived only in a weaker form than the agent filed them, and are recorded at the weaker severity.

Scope

Overall Risk	Low
Reviewed PRs	isolated-pools#559 (spoke pool), #562 (Sherlock fixes), #565 (Bailsec fixes) (stacked, each on the one before)
Reviewed Commit	c676d94583335302aa6b0aa14ddefc0107ba82b4 the tip of #565 , so it carries all three audit-fix stacks
Fix Commits	isolated-pools a9e07fd on feat/bailsec-audit (#565) venus-liquidity-hub 704d8f3 on feat/hub-spoke-integration (#22)
Size	5 contracts, 1,710 nSLOC

Contracts in scope

No.	Contract	Lines	Verdict	Findings
1	AdapterSpokeV1.sol	445	Medium	M-01, L-01, L-02, L-03
2	SpokeComptroller.sol	1,967	Informational	I-17 and 10 others
3	SpokePoolLens.sol	879	Informational	I-14, I-15
4	SpokeComptrollerInterface.sol	366	–	–
5	SpokeComptrollerStorage.sol	188	Informational	I-12
6	deploy/028-deploy-spoke-vtokens.ts	–	Low	L-04

[venus-liquidity-hub](#) is **not** in scope. Six informational items terminate at that boundary, and the open item M-01 has to be fixed on that side.

Summary of findings

Severity	Count
Critical	0
High	0
Medium	1
Low	4
Informational	18
Total	23

No critical or high severity issue was found. No unprivileged attacker can profit from any finding in this report. Nothing was found in the `SpokeComptroller` liquidation or liquidity logic itself; every Medium and Low sits in `AdapterSpokeV1` or in a deploy script.

Status at a glance

ID	Finding	Severity	Status
M-01	<code>withdraw</code> strands unredeemable dust on a low-decimal market	Medium	Open
L-01	<code>spotAPYBps</code> reverts on an emptied market	Low	Fixed
L-02	<code>maxDeposit</code> accrual margin is sized against the wrong quantity	Low	ACK
L-03	<code>deposit</code> can mint zero vTokens and still report success	Low	ACK
L-04	Spoke markets can be left owned by an EOA on a live network	Low	ACK
I-01 to I-18	See the informational table	Informational	ACK

Medium

[M-01] withdraw strands unredeemable dust on a market whose exchange rate is below 1e18

Contract	AdapterSpokeV1.sol , withdraw and maxWithdraw
Severity	Medium
Status	Open. The fix belongs in venus-liquidity-hub , which is outside this scope

Finding

A market lists at an exchange rate of $1e(18 + \text{underlyingDecimals} - 8)$. Both markets on the bsctestnet spoke are 6-decimal tokens (USDT and USDC), so the rate is $1e16$. At that scale one vToken unit is worth less than one base unit of the underlying, and 100 vToken units are worth 1.

Withdrawing rounds in two directions that do not meet. `maxWithdraw` reports `floor(balance x rate / 1e18)` , while the burn is `ceil(amount x 1e18 / rate)` . The leftover is 0 to 99 vToken units, and those are worth **zero** underlying.

After a full drain the position reads: `maxWithdraw 0`, `totalAssets 0`, `receiptBalance 99`. Retrying reverts, because the burn for 1 unit is 100 vTokens and only 99 are held. The market is empty so its rate is frozen and will never grow the 100x needed to clear the remainder.

The consequence is not the dust itself. `YieldGroup.removeResource` refuses to deregister a resource whose receipt balance is non-zero, so the market can never be retired:

```
// YieldGroupBase.sol
uint256 held = IResourceAdapter(cfg.adapter).receiptBalance(resource, address(this));
if (held != 0) revert ResourceHasBalance(resource, held);
```

The adapter's own NatSpec claims the opposite, that flooring "keeps the last vToken withdrawable, which is what lets a market be drained to zero and deregistered". That holds at 18 decimals, where the rate is $1e28$ and the remainder is always 0. It does not hold below 8.

What we did

TBD

Low

[L-01] spotAPYBps reverts on an emptied market instead of reporting zero

Contract	AdapterSpokeV1.sol , spotAPYBps
----------	---------------------------------

Severity	Low
----------	-----

Status	Fixed
--------	-------

Finding

The interest rate model ends by dividing income by the market's supply:

```
// JumpRateModelV2.getSupplyRate
uint256 supply = cash + borrows + badDebt - reserves;
return incomeToDistribute / supply;
```

There is no zero guard. Once a market is drained that divisor is zero and the read reverts with Panic 0x12. `spotAPYBps` called `supplyRatePerBlock()` unguarded, so any Hub read of a registered-but-emptied market reverted instead of reporting 0% APY.

The sibling lens already guards the identical read, three files away:

```
// SpokePoolLens.spokeVTokenMetadata
supplyRatePerBlockOrTimestamp: vToken.totalSupply() > 0 ? vToken.supplyRatePerBlock()
: 0,
```

What we did

Added the same guard the lens uses:

```
function spotAPYBps(address resource, uint256) external view override returns (uint64)
{
    if (IVTokenIsolated(resource).totalSupply() == 0) return 0;
```

A zero `totalSupply` is a safe proxy for a zero divisor. In the one case where the two differ, a donation leaving cash in a market with no supply, `borrows` is zero so the income is zero and 0 is the correct answer anyway.

The fix went into both copies of the adapter. The file under `isolated-pools/contracts/test/HubSpoke/` is a source copy rather than an npm import, so the fork suite would otherwise have kept testing the old code.

- Hub: [704d8f3](#) on [venus-liquidity-hub#22](#)
- Spoke: [a9e07fd](#) on [isolated-pools#565](#)

Three regression tests were added and confirmed to be real by stripping the guard and observing the Panic. Two pre-existing tests in `Views.t.sol` were reading the APY on an empty market and had to be given a funded market first; one of those two had been passing vacuously on `0 == 0`.

[L-02] maxDeposit accrual margin is sized against the wrong quantity

Contract	AdapterSpokeV1.sol , maxDeposit
----------	---------------------------------

Severity	Low
----------	-----

Status	ACK
--------	-----

Finding

maxDeposit withholds a small margin so that interest accruing between the read and the mint cannot push the deposit over the supply cap:

```
room -= room / CAP_TRIM_DIVISOR;    // 0.1% of the LEFTOVER room
```

What the margin has to absorb is the growth of `totalSupply x exchangeRate` , an absolute amount driven by the size of the whole loan book. The two quantities are unrelated, so on a market that is nearly full the margin is far too small.

6-decimal USDT, `totalBorrows` of 80,000,000 at 6.25% APY, 3 seconds, 25% reserve factor:

```
accrual over 3s = 356,748      units
leftover room   = 100,000,000 units
trim withheld   = 100,000      units   (3.5x too small)
at execution    = cap exceeded, mint reverts
```

The gap widens as the room shrinks. In the opposite direction, on a market with a large cap the same formula withholds far more than any accrual needs.

What we did

Accepted. No funds are at risk: the deposit leg reverts and the Hub's routing cascade catches that revert and routes around the market, which is exactly the backstop the trim exists to supplement.

The 0.1% trim is the pattern inherited from `AdapterCoreV1` , where it has the same shape. Changing it here alone would make the two adapters disagree for no gain. If it is revisited, the right form is to size the margin off the pending accrual, or to state the same-transaction `accrue(resource)` precondition that `maxWithdraw` already states for itself.

[L-03] deposit can mint zero vTokens and still report success

Contract	AdapterSpokeV1.sol , deposit
----------	------------------------------

Severity	Low
----------	-----

Status	ACK
--------	-----

Finding

`deposit` refuses an amount too small to buy one vToken. It reads the exchange rate to size that floor, but it reads it **before** `mint` runs `accrueInterest`, while the actual mint divides by the rate *after* the accrual.

The two differ whenever the pre-accrual rate is an exact multiple of $1e18$, which leaves the ceiling no slack:

```
rate read      = 10,000,000,000 (1e28)
floor computed = 1e10
rate at mint   = 10,000,000,001 (one unit higher)
vTokens minted = floor(1e10 x 1e18 / (1e28 + 1)) = 0
```

The market keeps the underlying, mints nothing, and `deposit` still returns the full amount as deposited.

What we did

Accepted. Nothing is lost.

The underlying is not destroyed. It stays in the market as cash and raises the exchange rate for every existing holder. A spoke market is funded only by the Hub, including the initial supply required at listing, which the listing VIP mints on the Hub's behalf, so the Hub is the only holder. The value therefore returns to the Hub's own position in the same block.

The exposure is also bounded at one vToken unit per call and cannot be triggered by an unprivileged account.

A balance-comparison fix was written and then dropped from the PR, because the same gap is not specific to this adapter. `AdapterCoreV1.deposit` has it with no floor at all, and the Flux and FRV adapters discard the `ERC4626 shares` return value. It belongs in a change of its own that covers all of them.

[L-04] Spoke markets can be left owned by an EOA on a live network

Contract	deploy/028-deploy-spoke-vtokens.ts:79-83
----------	--

Severity	Low
----------	-----

Status	ACK
--------	-----

Finding

The script picks the vToken owner by falling back to the deployer when no timelock is configured, and prints that choice as ordinary output:

```
const vTokenOwner =
  hre.network.live && preconfiguredAddresses.NormalTimelock
    ? await toAddress(preconfiguredAddresses.NormalTimelock)
    : deployer;
console.log(`Markets will be owned by ${vTokenOwner}`);
```

Every sibling fails loudly instead. The isolated-pools equivalent at 009:28 resolves through `deployments.get("NormalTimelock")` and **throws** when it is absent, and 024:27 , 025:33 and 027:37 each print `WARNING: no NormalTimelock configured` . Only 028 is silent.

What we did

Accepted. The mainnet listing runs through a VIP that sets market ownership explicitly, so the fallback is never the final state. VToken is Ownable2Step , so even if it were reached the position is recoverable.

This holds only while the listing VIP sets ownership explicitly, since the script itself gives no signal that the fallback was taken. Adding the same one-line warning the three sibling scripts already print would remove the dependency.

Informational

Trails with a real code smell where no exploit path could be completed. Every row was checked against the source; the Assessment column states what that check found. All 18 are acknowledged and none is actioned.

ID	Where	Observation and assessment
I-01	<code>_safeGetUnderlyingPrices</code>	The collateral factor check prices at the bounded window minimum, the liquidation check at spot. Neither attack on the split works: the bounded price follows a crash exactly, and an account blocked from borrowing but not yet liquidatable is solvent at spot. ACK: no change.

I-02	<code>unlistMarket</code>	Requires every action paused and the caps and collateral factor at zero, but not that the market is empty. <code>getAssetsIn</code> skips unlisted markets, so a borrower's debt stops counting and their other collateral walks out. Governance-only and upstream-identical. ACK. Confirm a market has no open borrows before unlisting it.
I-03	<code>enterMarketForAccount</code>	A YieldGroup has no route to <code>exitMarket</code>, so the membership is permanent, and it makes every redeem run the liquidity check. That check reverts only while <code>deviationBoundedOracle</code> is unset, which the owner can set at any time. A YieldGroup carries no debt. ACK: no change.
I-04	<code>validateRegistration</code>	Reads the comptroller out of the thing it is checking, <code>resource.comptroller()</code>, so a fake resource can answer yes to both checks. It catches misconfiguration, not a hostile contract. Contained: <code>YieldGroupBase.addResource</code> is ACM-gated. ACK: no change.
I-05	<code>maxDeposit</code> , <code>validateRegistration</code>	<code>deposit</code> and <code>withdraw</code> enforce <code>onlyDelegateCall</code>; these two views do not, and read <code>msg.sender</code> as the supplier. Nothing to exploit: both are views, neither writes state, and another caller only learns the answer for its own address. ACK: no change.
I-06	<code>preMintHook</code>	The allowlist meters the credited account, not the payer, and <code>mintBehalf</code> is permissionless, so anyone can push dust onto an allowlisted YieldGroup and keep <code>removeResource</code> reverting. No funds at risk. Rules out fixing M-01 on the withdraw side. ACK. Folded into M-01.
I-07	<code>totalAssets</code> vs <code>maxDeposit</code>	The market mints at a rate that keeps <code>badDebt</code> in, while <code>totalAssets</code> marks it out, so a deposit into a market with bad debt is written down the same block. No loss: the markdown on the new vTokens is exactly the markup on the old, and the Hub holds both. ACK: no change.
I-08	<code>accrue</code>	The only mutating function without <code>onlyDelegateCall</code>, and it calls <code>accrueInterest()</code> on any address given. Deliberate, since that moves the market's global index. No impact: the adapter holds no storage, role or standing allowance. ACK: no change.
I-09	<code>maxWithdraw</code>	Bounds the withdrawal by <code>cash - totalReserves</code> from stored state, while <code>redeem</code> accrues first and checks against less free cash. It reverts, no loss, and every Hub routing path runs <code>_accrueFees()</code> on entry, so the bound is current. ACK: no change.

I-10	<code>setCollateralFactor</code>	Validates the price on the resilient oracle, but the check it guards prices through <code>deviationBoundedOracle</code>. If that one is unset or reverting, the collateral factor sets fine while borrow, withdraw and exit revert. Never misprices. Inherited from upstream. ACK: no change.
I-11	<code>healAccount</code>	With no collateral to take, <code>healAccount</code> never reaches <code>seize</code>, so the self-liquidation guard never runs. Harmless: <code>maxClearableDebt</code> is 0, so the repayment is 0% and there is no discount to capture. The liquidation allowlist still applies. ACK: no change.
I-12	<code>_accumulateMarketPosition</code>	<code>maxClearableDebt</code> is accumulated only under the liquidation-threshold weighting and stays 0 under the collateral factor, where a caller would put <code>healAccount</code> on a 0% repayment. Every caller passes the right weighting today. Documented, not enforced. ACK: no change.
I-13	<code>_safeGetUnderlyingPrices</code>	The oracle stores its window as <code>uint128</code>, and <code>_updateProtectionStates</code> runs over every market the account entered, so one out-of-range price reverts borrow, redeem and exit account-wide. Only a 0-decimal token above about \$340 reaches it, and it reverts rather than misprices. ACK: no change.
I-14	<code>_calculateNotDistributedAwards</code>	Takes the reward basis, and the current block or timestamp, from the lens's own <code>isTimeBased</code> rather than the <code>RewardsDistributor</code>'s, so mismatched bases subtract a block number from a timestamp. View-only, so the effect is a wrong preview. ACK: no change.
I-15	<code>spokeVTokenMetadata</code>	<code>exchangeRate</code>, <code>totalBorrows</code> and <code>totalReserves</code> are accrued to this block, while <code>supplyRatePerBlockOrTimestamp</code> beside them is read from stored state, so it lags. View-only, a slightly stale rate in the UI. ACK: no change.
I-16	<code>unlistMarket</code>	Clears <code>isListed</code> and nothing else. Pauses, caps, the supply allowlist, the per-market incentive and <code>isForcedLiquidationEnabled</code> all survive a relist; only the collateral factor and threshold reset. Left on, the last one lets liquidators hit healthy borrowers. ACK. A relist runbook must confirm it is off.
I-17	<code>setLiquidationIncentive</code>	Downgraded from Medium. Enforces <code>incentive >= 1.05e18</code>, 1 plus the default 5% seize share, not each market's live share. With share 8% and incentive 1.05 a liquidator nets \$97 on a \$100 repay. Governance sequencing only, and a per-market check does not fit in 24,576 bytes. ACK. A market with a raised seize share must get its own incentive.

I-18 `totalAssets`

Downgraded from Medium. `maxWithdraw` returns 0 when REDEEM is paused, `totalAssets` has no such check, so an unlisted market still marks at full value in the Hub's NAV. Unreachable while live: `unlistMarket` needs all nine actions paused.

ACK. Withdraw the Hub's position and remove the resource before any unlist.
